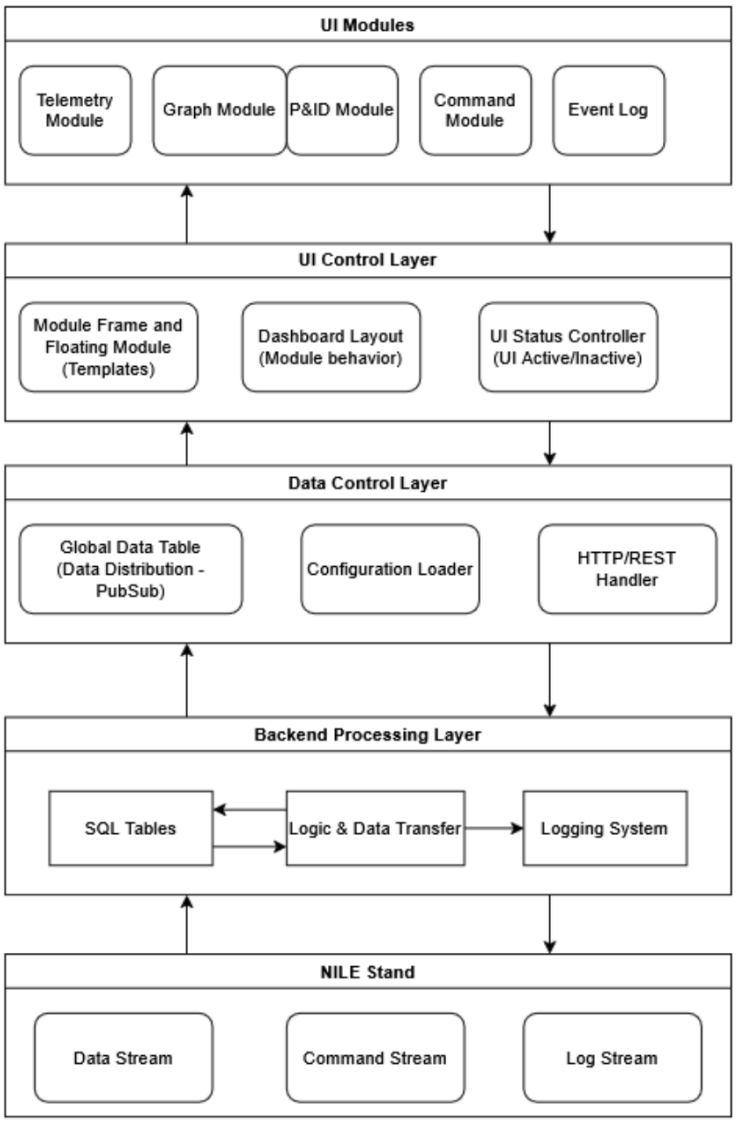
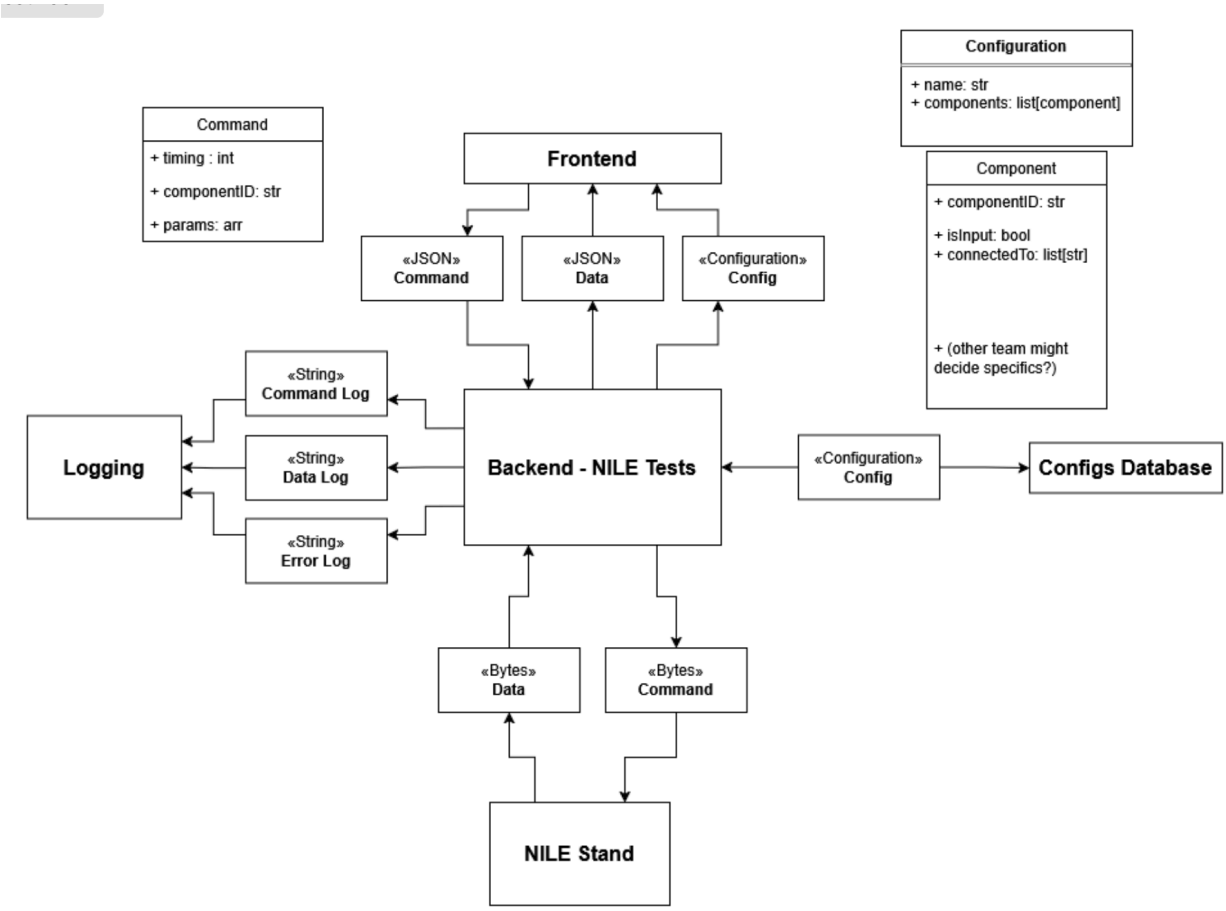
1. Primary Architecture

We are using a layered architecture, which lets our application split a complex data transfer into several steps. Thus, we can represent how commands and received data travel through the system, often between the NILE stand and the User Interface Modules.

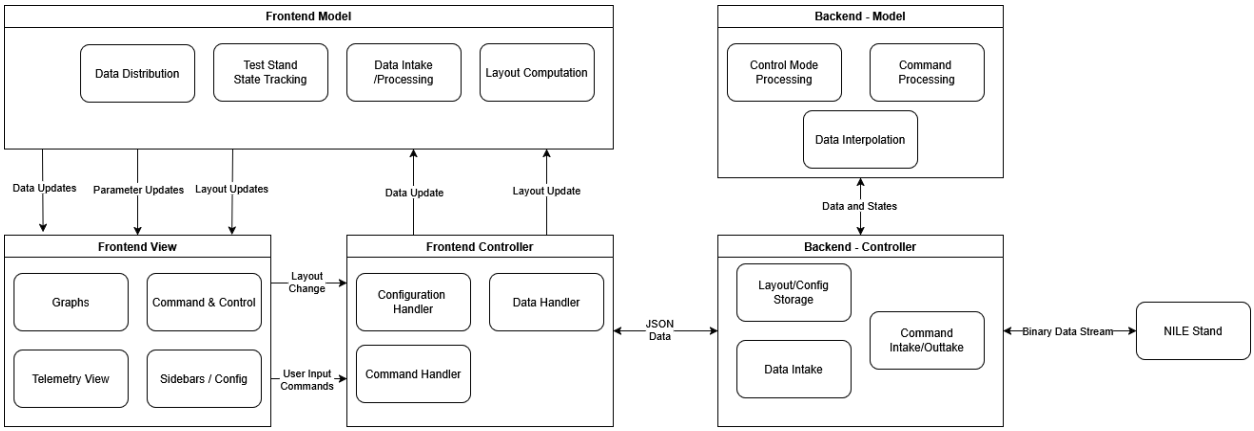


2. High-Level Design Diagram

This first model shows the main data objects moving between our components.

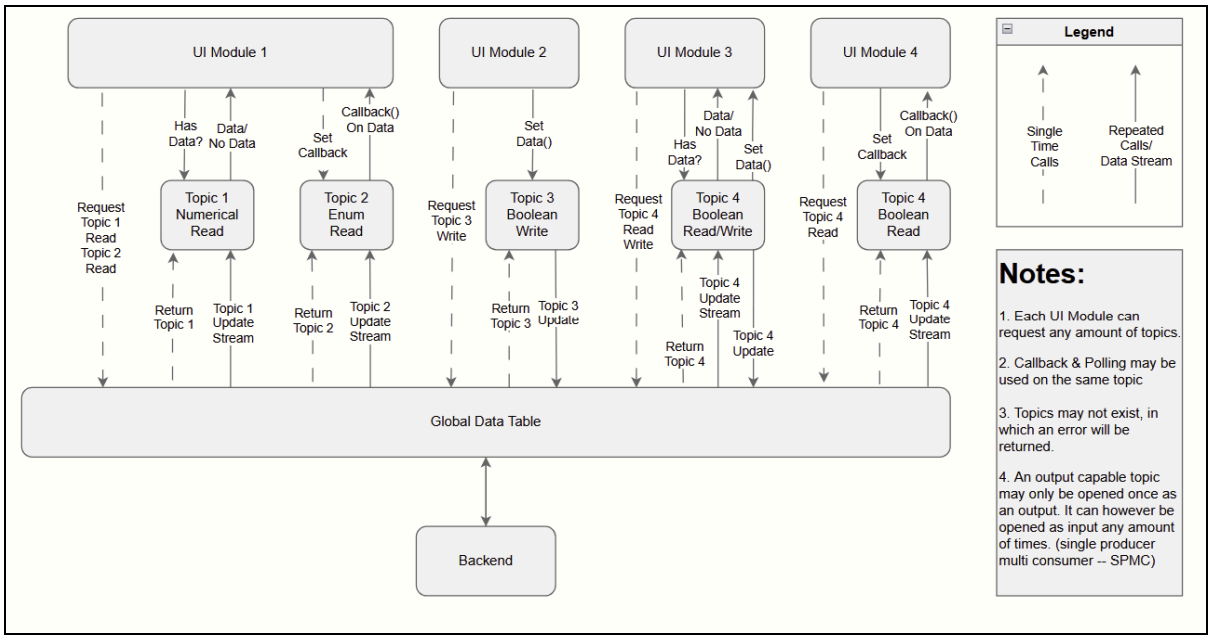


We also have this alternate design which focuses more on the MVC pattern. As such, it better shows specific components within our Frontend-backend system.



3. Design Pattern

We have a model for the observer pattern. Our global data table publishes updates to all UI modules, which allows us to track current data without needing to send to each active module individually. This pattern will primarily happen in the frontend, where the data table and modules are found.



This pattern works better than flat code because it will scale better as we account for more methods of collecting and displaying code.

4. Design Decisions & Tradeoffs

One decision we made was to store the configuration data in single SQL cells as large JSONs, rather than have our configs reference a separate table of components. We chose to do this because components are unlikely to be used individually, and thus we will be sending and receiving entire configurations regardless of how we store them. This makes our implementation a rare case where it is both more space-efficient and equally time-efficient to store these lists of objects in a single table.

The major tradeoff is that it will make our backend more implementation-heavy, as our SQL tables require more code to interact with. They are also much more complex to communicate with than, for instance a CSV file, which means we must spend more time developing our backend communication.

Another design decision was to separate the frontend into independent UI modules, instead of having everything be integrated together. We chose to do this because it allows us to have easy separation of responsibilities between each UI module / component. This means that we can work on different parts of the program without needing to worry about how another part of the code would impact us. This also allows us to collaborate effectively with DE-NILE, which is not part of our class session.

The tradeoff is that each UI module will have some amount of redundant code. The software will also be slightly more abstracted by layers that are needed for self-contained UI modules to work. This means that reasoning through the software, across layers, will be more complicated. We believe these trade-offs are reasonable to make considering how much it helps with project management and development setup.